Modeling

```
library(MASS)
library(dplyr)
library(ggplot2)
library(faraway)
library(lmtest)
library(randomForest)

theme_set(theme_minimal(base_size = 11) +
            theme(plot.title    = element_text(hjust = 0.5),
                  plot.subtitle = element_text(hjust = 0.5)))

tech <- readRDS("../data/processed/tech_clean.rds")
```

1. Model Data

Overlapping Features

```
# Exact overlaps: these add up to 1 for every article
all(abs(tech$rate_positive_words + tech$rate_negative_words - 1) < 1e-9)
```

```
## [1] TRUE
```

```
all(abs(tech$LDA_00 + tech$LDA_01 + tech$LDA_02 + tech$LDA_03 + tech$LDA_04 - 1) < 1e-6)
```

```
## [1] TRUE
```

- `rate_negative_words` is exactly 1 minus `rate_positive_words`
- `LDA_04` is exactly 1 minus the other four topic scores
- Keeping all of them would make the model unsolvable (exact collinearity)

```
overlap_pairs <- data.frame(
  dropped = c("n_unique_tokens", "self_reference_min_shares", "self_reference_max_shares",
              "kw_max_avg"),
  kept    = c("n_non_stop_unique_tokens", "self_reference_avg_shares",
              "self_reference_avg_shares", "kw_avg_avg")
)

overlap_pairs$correlation <- round(mapply(function(a, b) cor(tech[[a]], tech[[b]]),
                                          overlap_pairs$dropped, overlap_pairs$kept), 2)
knitr::kable(overlap_pairs, row.names = FALSE)
```

dropped	kept	correlation
n_unique_tokens	n_non_stop_unique_tokens	0.93
self_reference_min_shares	self_reference_avg_shares	0.83
self_reference_max_shares	self_reference_avg_shares	0.88
kw_max_avg	kw_avg_avg	0.75

- Each dropped column has a correlation of 0.75 to 0.93 with the one we keep

```
# VIF with every candidate count, rate and popularity feature
candidates <- tech |>
  select(n_tokens_content, num_hrefs, num_self_hrefs, n_unique_tokens,
         n_non_stop_unique_tokens, global_sentiment_polarity,
         global_rate_positive_words, global_rate_negative_words, rate_positive_words,
         kw_min_min, kw_max_min, kw_avg_min, kw_min_max, kw_max_max, kw_avg_max,
         kw_min_avg, kw_max_avg, kw_avg_avg, self_reference_min_shares,
         self_reference_max_shares, self_reference_avg_shares)

round(sort(vif(candidates), decreasing = TRUE)[1:6], 1)
```

```
## self_reference_avg_shares self_reference_max_shares                kw_max_min
##                       28.6                       12.2                      11.8
##                 kw_avg_min           n_unique_tokens       rate_positive_words
##                       11.4                       10.6                       8.8
```

```
kept <- tech |>
  select(n_tokens_content, num_hrefs, num_self_hrefs, n_non_stop_unique_tokens,
         global_sentiment_polarity, global_rate_positive_words,
         global_rate_negative_words, kw_avg_avg, self_reference_avg_shares)

round(sort(vif(kept), decreasing = TRUE), 1)
```

```
##  global_sentiment_polarity           n_tokens_content
##                        2.3                        2.3
##   n_non_stop_unique_tokens                  num_hrefs
##                        1.9                        1.8
## global_rate_positive_words global_rate_negative_words
##                        1.7                        1.7
##             num_self_hrefs                  kw_avg_avg
##                        1.5                        1.0
##  self_reference_avg_shares
##                        1.0
```

- Before dropping, five features have VIF above 10. The highest is `self_reference_avg_shares` (28.6), because its min and max versions are also in
- After dropping, every VIF is 2.3 or lower
- The nine `kw_` columns are all summaries of keyword popularity, and the min columns are full of `-1` and `0` placeholders. We keep only `kw_avg_avg`, the average keyword's average shares, as one clear measure

Features Used

```
model_data <- tech |>
  mutate(
    log_words      = log1p(n_tokens_content),
    log_hrefs      = log1p(num_hrefs),
    log_self_hrefs = log1p(num_self_hrefs),
    log_kw_avg_avg = log1p(kw_avg_avg),
    log_self_ref   = log1p(self_reference_avg_shares),
    quarter        = factor(quarter)
  )

controllable <- c("weekend", "vid_group", "img_group", "log_words", "n_tokens_title",
                  "log_hrefs", "log_self_hrefs", "num_keywords", "average_token_length",
                  "n_non_stop_unique_tokens", "global_sentiment_polarity",
                  "global_subjectivity", "global_rate_positive_words",
                  "global_rate_negative_words", "avg_positive_polarity",
                  "avg_negative_polarity", "title_sentiment_polarity", "title_subjectivity")

uncontrollable <- c("log_kw_avg_avg", "log_self_ref", "LDA_00", "LDA_01", "LDA_02", "LDA_03",
             "quarter")

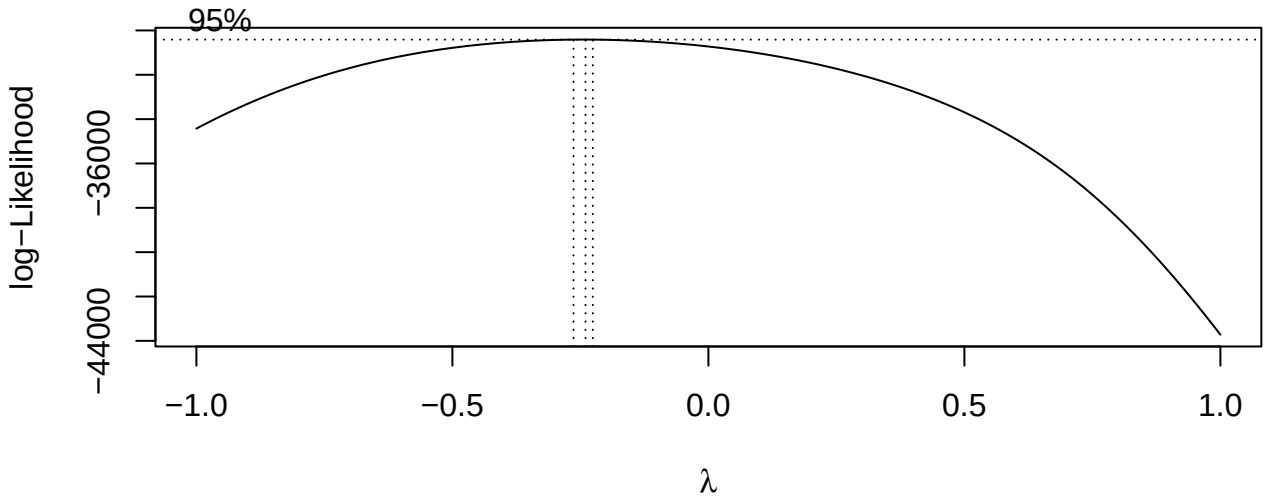
features <- paste(c(controllable, uncontrollable), collapse = " + ")
```

- Controllable features: things an editor chooses (day, visuals, length, links, wording, tone)
- Uncontrollable features: things an editor can't directly change (topic popularity, popularity of linked articles, topic, time period)
- Skewed counts use `log(x + 1)`
- `quarter` replaces `timedelta` to account for drift over time
- Dropped for overlapping with a kept feature (see Overlapping Features above)

2. Full Linear Model

Response Transformation

```
g_raw <- lm(as.formula(paste("shares ~", features)), data = model_data)
bc <- boxcox(g_raw, lambda = seq(-1, 1, by = 0.01))
```




```
bc$x[which.max(bc$y)]
```

```
## [1] -0.24
```

```
range(bc$x[bc$y > max(bc$y) - qchisq(0.95, 1) / 2])
```

```
## [1] -0.26 -0.23
```

- Box-Cox picks $\lambda \approx -0.24$, far from 1, so raw shares need a transformation
- We use log shares ($\lambda = 0$) instead. It's close to the best λ and each coefficient reads as a percent change in shares

Weekday vs Weekend

```
g_full <- lm(as.formula(paste("log_shares ~", features)), data = model_data)
g_day  <- update(g_full, . ~ . - weekend + weekday)

anova_tab <- anova(g_full, g_day)
rownames(anova_tab) <- c("Weekday vs weekend", "7 days")
knitr::kable(anova_tab, digits = 3)
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
Weekday vs weekend	7289	4399.253	NA	NA	NA	NA
7 days	7284	4393.926	5	5.327	1.766	0.116

- H_0: the six extra day terms are 0 (weekday vs weekend is enough)
- p = 0.12, so we fail to reject and keep weekday vs weekend

Collinearity

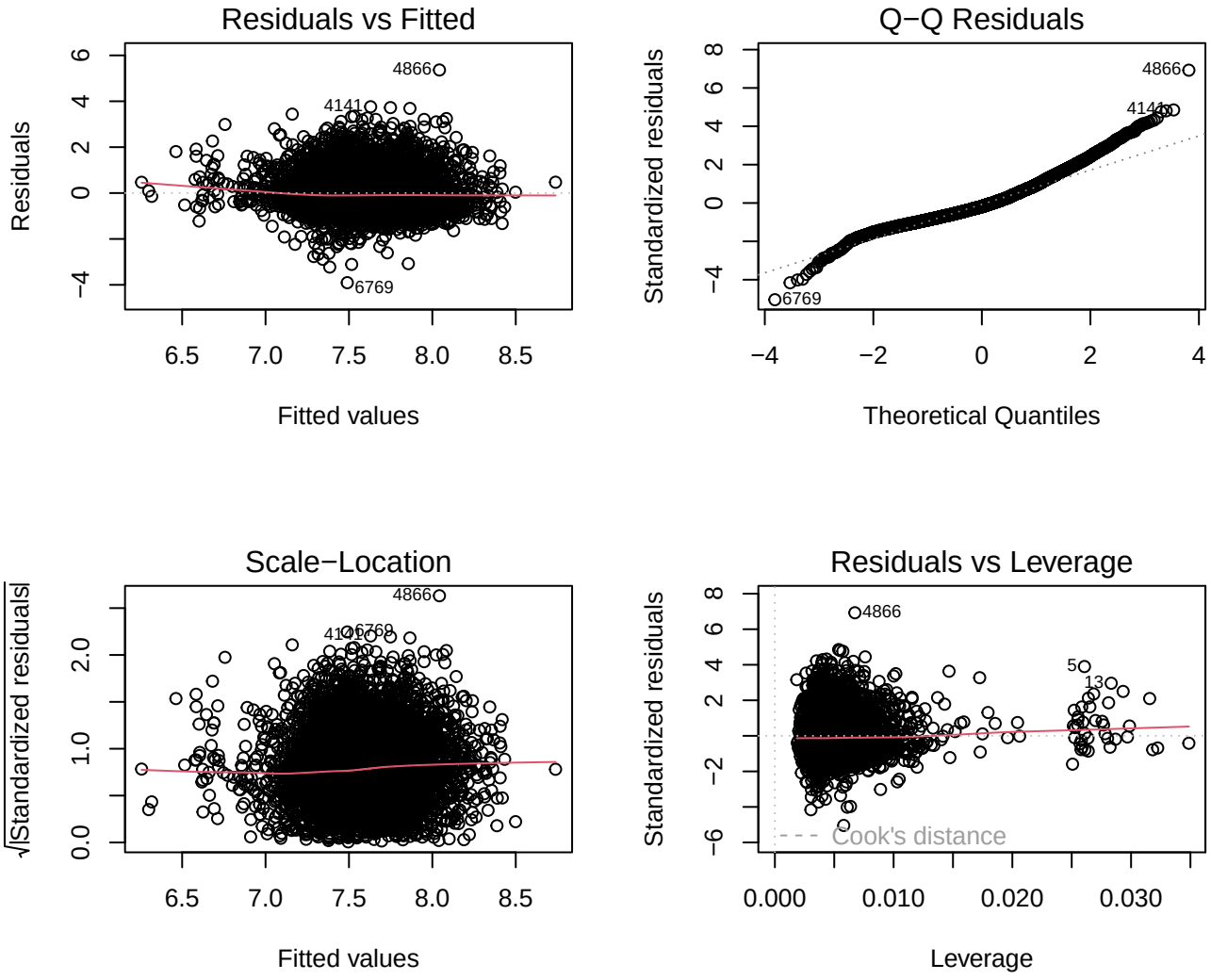
```
round(sort(vif(g_full), decreasing = TRUE)[1:6], 2)
```

```
##  global_sentiment_polarity                  log_words
##                       6.07                       3.40
##              img_group11+   n_non_stop_unique_tokens
##                       3.19                       3.06
##                 img_group1 global_rate_negative_words
##                       2.65                       2.59
```

- All VIFs are under 10, so collinearity isn't a problem
- The highest is article tone (6.07), since the positive and negative word rates partly measure the same thing

Diagnostics

```
par(mfrow = c(2, 2))
plot(g_full)
```




```
par(mfrow = c(1, 1))
```

- Residuals vs Fitted: centered on 0 with no clear curve
- Q-Q: heavy upper tail from a few viral articles
- Scale-Location: close to flat
- Residuals vs Leverage: no points past the Cook's distance lines

```r
bptest(g_full)
```

```
##
##  studentized Breusch-Pagan test
##
## data:  g_full
## BP = 171.22, df = 35, p-value < 2.2e-16
```

- H_0: constant error variance. p < 0.001, so we reject
- With 7,325 articles, the test picks up small differences in spread
- Coefficient estimates are still unbiased, but p-values may be a little too small.

Unusual Points

```
n <- nrow(model_data)
p <- length(coef(g_full))

lev <- hatvalues(g_full)
sum(lev > 2 * p / n)
```

```
## [1] 147
```

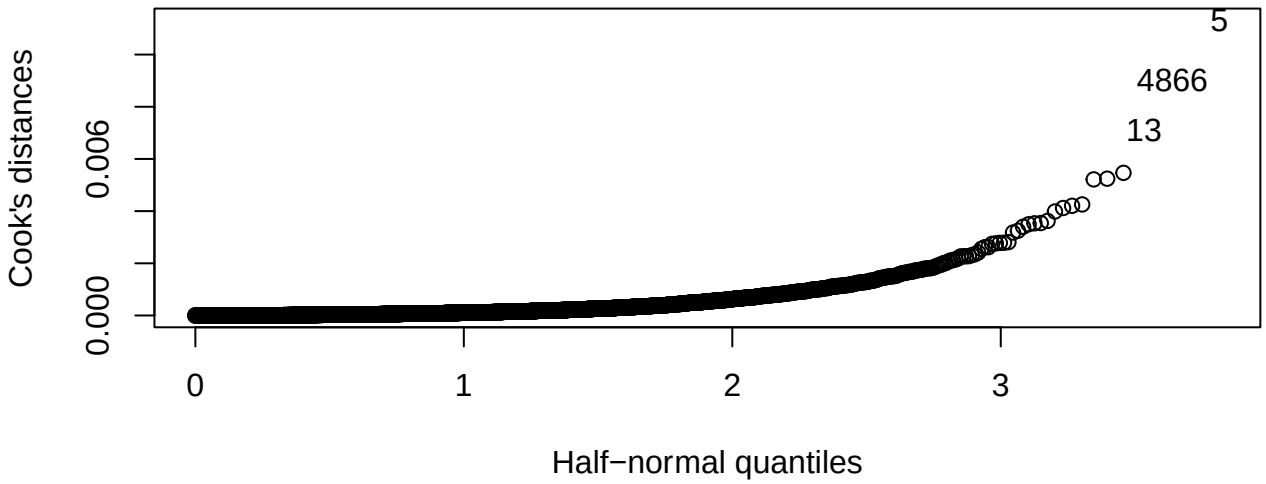
```
jack <- rstudent(g_full)
sum(abs(jack) > abs(qt(0.05 / (2 * n), n - p - 1)))  # Bonferroni
```

```
## [1] 5
```

```
cook <- cooks.distance(g_full)
max(cook)
```

```
## [1] 0.0113109
```

```
halfnorm(cook, 3, ylab = "Cook's distances")
```



- 147 articles have high leverage ($h_i > 2p/n$), 2% of the data
- 5 articles are outliers after the Bonferroni correction, all viral articles with far more shares than predicted
- Largest Cook's distance is 0.011, far below 1, so no single article changes the fit. We keep all articles

3. Variable Selection

```
g_aic <- step(g_full, direction = "backward", trace = 0)
g_bic <- step(g_full, direction = "backward", k = log(n), trace = 0)

all_terms <- c(controllable, uncontrollable)

data.frame(
  feature = all_terms,
  group   = ifelse(all_terms %in% controllable, "Controllable", "Uncontrollable"),
  AIC     = ifelse(all_terms %in% attr(terms(g_aic), "term.labels"), "Kept", ""),
  BIC     = ifelse(all_terms %in% attr(terms(g_bic), "term.labels"), "Kept", "")
) |> knitr::kable(row.names = FALSE)
```

feature	group	AIC	BIC
weekend	Controllable	Kept	Kept
vid_group	Controllable	Kept	Kept
img_group	Controllable	Kept	
log_words	Controllable	Kept	Kept
n_tokens_title	Controllable	Kept	
log_hrefs	Controllable	Kept	Kept
log_self_hrefs	Controllable	Kept	Kept
num_keywords	Controllable	Kept	
average_token_length	Controllable		
n_non_stop_unique_tokens	Controllable	Kept	
global_sentiment_polarity	Controllable	Kept	Kept
global_subjectivity	Controllable	Kept	Kept
global_rate_positive_words	Controllable	Kept	
global_rate_negative_words	Controllable		
avg_positive_polarity	Controllable		
avg_negative_polarity	Controllable		
title_sentiment_polarity	Controllable	Kept	Kept
title_subjectivity	Controllable		
log_kw_avg_avg	Uncontrollable	Kept	Kept
log_self_ref	Uncontrollable	Kept	Kept
LDA_00	Uncontrollable		
LDA_01	Uncontrollable	Kept	
LDA_02	Uncontrollable	Kept	
LDA_03	Uncontrollable		
quarter	Uncontrollable	Kept	Kept

- Backward elimination, starting from the full model
- AIC drops 7 features. BIC drops 14, since it penalizes extra terms more
- Both keep weekend, videos, word count, links, links, article tone, subjectivity, headline tone, keyword popularity, linked-article popularity and quarter
- BIC also drops images, title length, number of keywords, word variety, the other sentiment measures and the topic scores

```
data.frame(
  model    = c("Full", "AIC", "BIC"),
```

```
  terms    = c(length(coef(g_full)), length(coef(g_aic)), length(coef(g_bic))) - 1,
  adj_r2   = round(c(summary(g_full)$adj.r.squared, summary(g_aic)$adj.r.squared,
                     summary(g_bic)$adj.r.squared), 4),
  AIC      = round(c(AIC(g_full), AIC(g_aic), AIC(g_bic))),
  BIC      = round(c(BIC(g_full), BIC(g_aic), BIC(g_bic)))
) |> knitr::kable(row.names = FALSE)
```

model	terms	adj_r2	AIC	BIC
Full	35	0.0853	17127	17382
AIC	28	0.0857	17116	17323
BIC	18	0.0826	17132	17270

- Adjusted R^2 barely changes (0.085 to 0.083), so the dropped features add almost nothing
- We use the BIC model as the final model since it's the simplest

Final Model

```
knitr::kable(summary(g_bic)$coefficients, digits = 4)
```

	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	5.7119	0.1567	36.4590	0.0000
weekendWeekend	0.2934	0.0278	10.5360	0.0000
vid_group1	0.1607	0.0229	7.0273	0.0000
vid_group2+	0.2597	0.0381	6.8144	0.0000
log_words	0.0963	0.0152	6.3526	0.0000
log_hrefs	0.1093	0.0196	5.5677	0.0000
log_self_hrefs	-0.1613	0.0183	-8.8107	0.0000
global_sentiment_polarity	-0.5630	0.1292	-4.3572	0.0000
global_subjectivity	0.4052	0.1356	2.9880	0.0028
title_sentiment_polarity	0.1274	0.0399	3.1948	0.0014
log_kw_avg_avg	0.1270	0.0161	7.9003	0.0000
log_self_ref	0.0432	0.0040	10.7213	0.0000
quarter2013 Q2	-0.2578	0.0355	-7.2662	0.0000
quarter2013 Q3	-0.3422	0.0352	-9.7177	0.0000
quarter2013 Q4	-0.2307	0.0378	-6.1047	0.0000
quarter2014 Q1	-0.1713	0.0383	-4.4741	0.0000
quarter2014 Q2	-0.1876	0.0374	-5.0161	0.0000
quarter2014 Q3	-0.2422	0.0385	-6.2885	0.0000
quarter2014 Q4	-0.3449	0.0393	-8.7774	0.0000

```
summary(g_bic)$r.squared
```

```
## [1] 0.08482677
```

- Every remaining term is significant at 0.01
- The final model explains about 8.5% of the variation in log shares

4. Random Forest

```
set.seed(443)
train_rows <- sample(n, round(0.8 * n))
train <- model_data[train_rows, ]
test  <- model_data[-train_rows, ]

rf <- randomForest(as.formula(paste("log_shares ~", features)),
                   data = train, ntree = 300, importance = TRUE)

lm_full_train <- update(g_full, data = train)
lm_bic_train  <- update(g_bic, data = train)
```

- The forest uses every feature, since it doesn't need variable selection

Accuracy on Test Articles

```
test_r2 <- function(pred) {
  1 - sum((test$log_shares - pred)^2) / sum((test$log_shares - mean(test$log_shares))^2)
}

data.frame(
  model   = c("Full linear", "BIC linear", "Random forest"),
  test_r2 = round(c(test_r2(predict(lm_full_train, test)),
                    test_r2(predict(lm_bic_train, test)),
                    test_r2(predict(rf, test))), 3)
) |> knitr::kable(row.names = FALSE)
```

model	test_r2
Full linear	0.078
BIC linear	0.079
Random forest	0.138

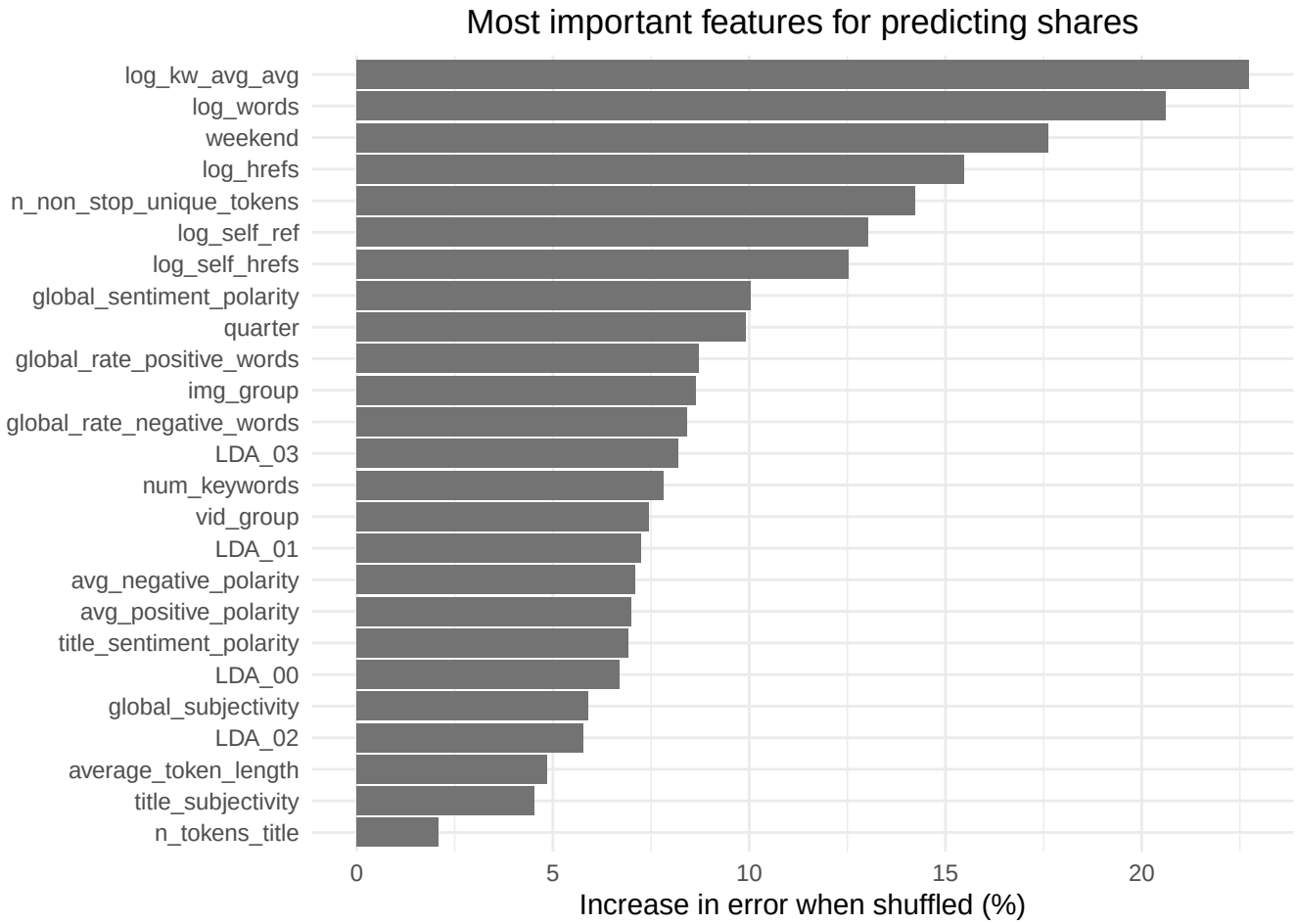
- The forest explains more on new articles (about 14% vs 8%), since it picks up curves and combinations of features
- The BIC model does as well as the full model with far fewer terms

5. What Drives Shares

```
ranking <- data.frame(feature = rownames(importance(rf)),
                      importance = importance(rf)[, "%IncMSE"]) |>
  mutate(group = ifelse(feature %in% controllable, "Controllable", "Uncontrollable"))
```

- Importance = how much worse the forest's predictions get when a feature's values are shuffled

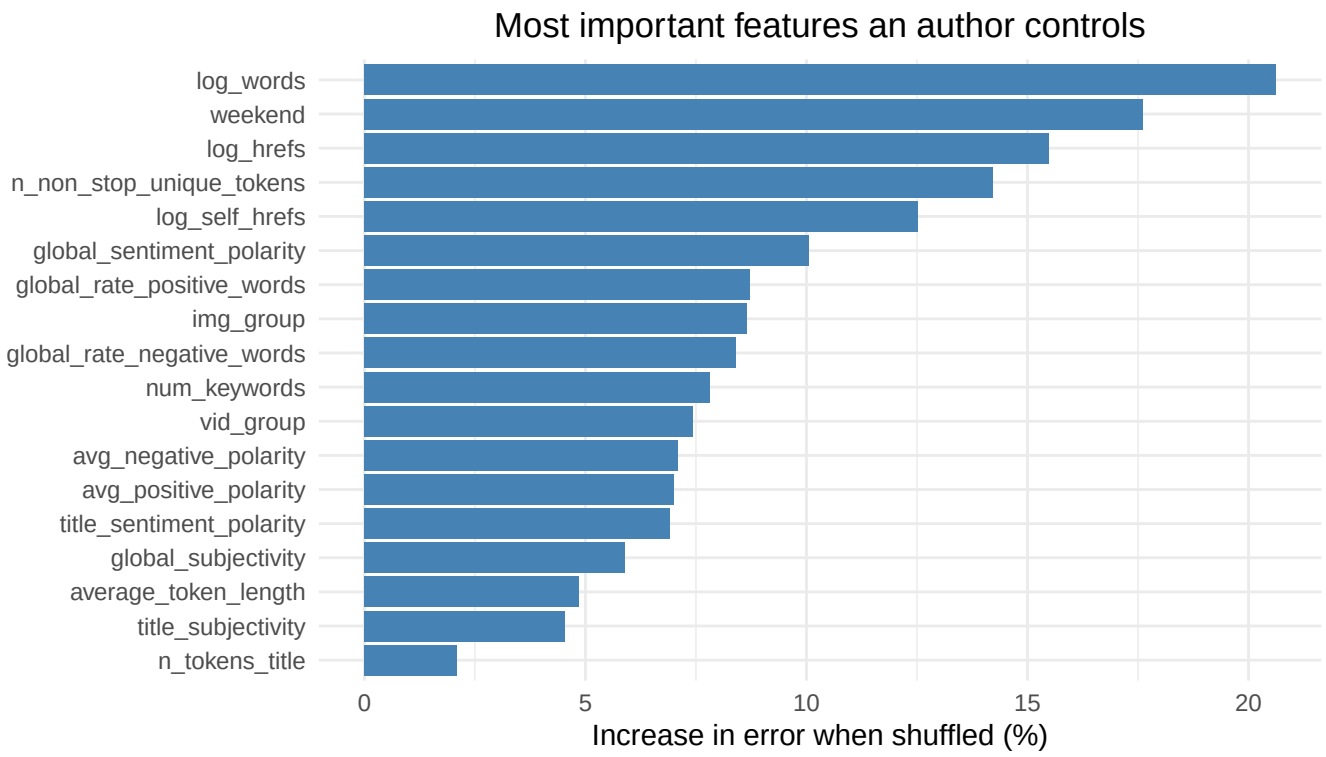
```
ggplot(ranking, aes(importance, reorder(feature, importance))) +
  geom_col(fill = "grey45") +
  labs(x = "Increase in error when shuffled (%)", y = NULL,
       title = "Most important features for predicting shares")
```



- Keyword popularity, word count, weekend and links rank highest

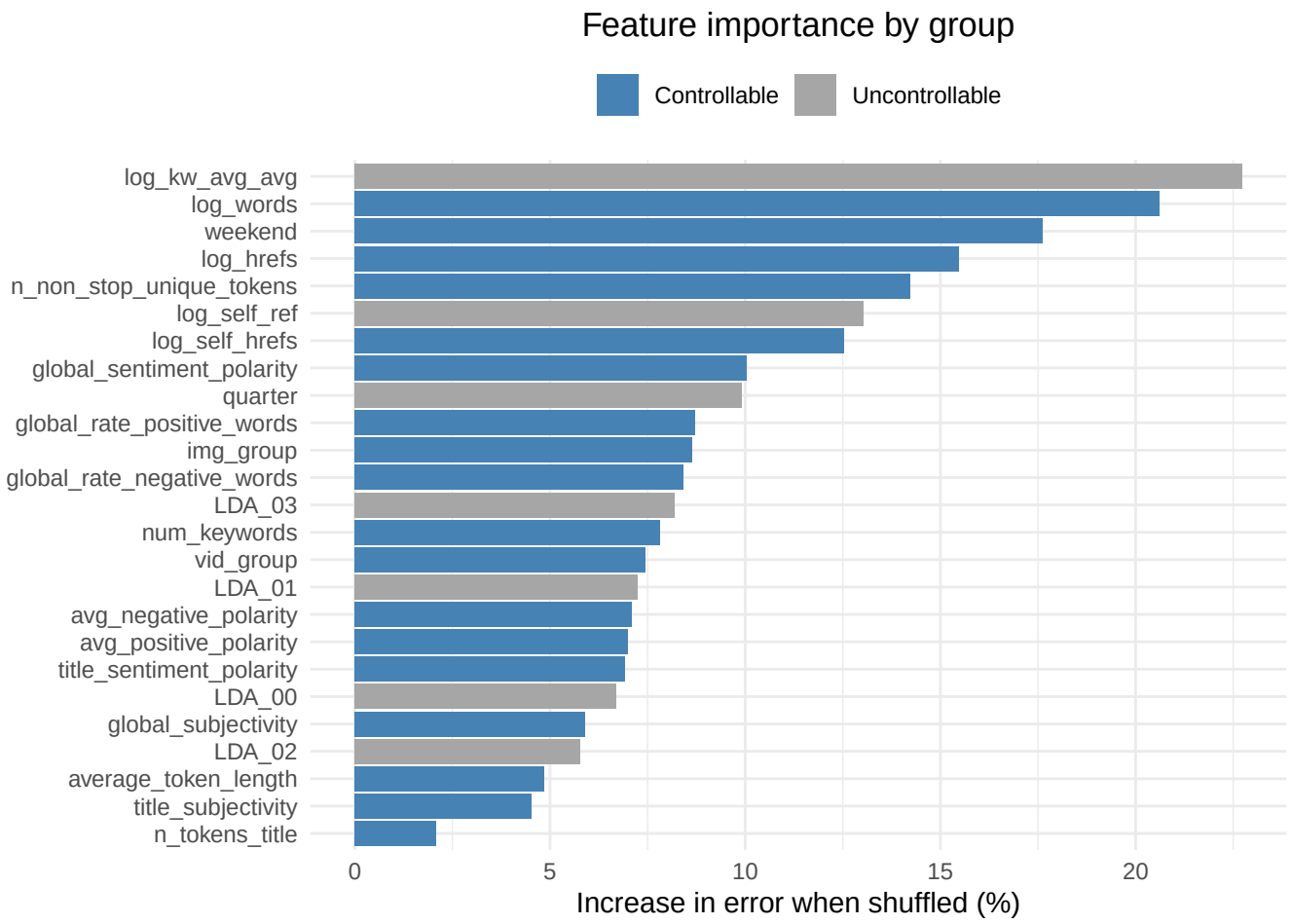
```
ranking |>
  filter(group == "Controllable") |>
  ggplot(aes(importance, reorder(feature, importance))) +
```

```
  geom_col(fill = "steelblue") +
  labs(x = "Increase in error when shuffled (%)", y = NULL,
       title = "Most important features an author controls")
```



- Word count, weekend and links rank highest among controllable features
- Videos rank lower than their effect suggests because 72% of articles have none, so shuffling them changes few predictions

```
ggplot(ranking, aes(importance, reorder(feature, importance), fill = group)) +
  geom_col() +
  scale_fill_manual(values = c(Controllable = "steelblue", Uncontrollable = "grey65"),
                    name = NULL) +
  labs(x = "Increase in error when shuffled (%)", y = NULL,
       title = "Feature importance by group") +
  theme(legend.position = "top")
```



- Forest importance and BIC selection mostly agree. The main exception is word variety (`n_non_stop_unique_tokens`), which ranks high in the forest but is dropped by BIC, likely because its effect isn't a straight line

6. Interpretation

- Effects come from the final (BIC) model, holding every other feature fixed
- Since the response is log shares, a coefficient $\hat{\beta}$ means shares change by $100(e^{\hat{\beta}} - 1)\%$ for a one-unit increase
- For log features, doubling the count changes shares by $100(2^{\hat{\beta}} - 1)\%$
- For scores from 0 to 1, we use a 0.1 increase: $100(e^{0.1\hat{\beta}} - 1)\%$

```
b <- coef(g_bic)

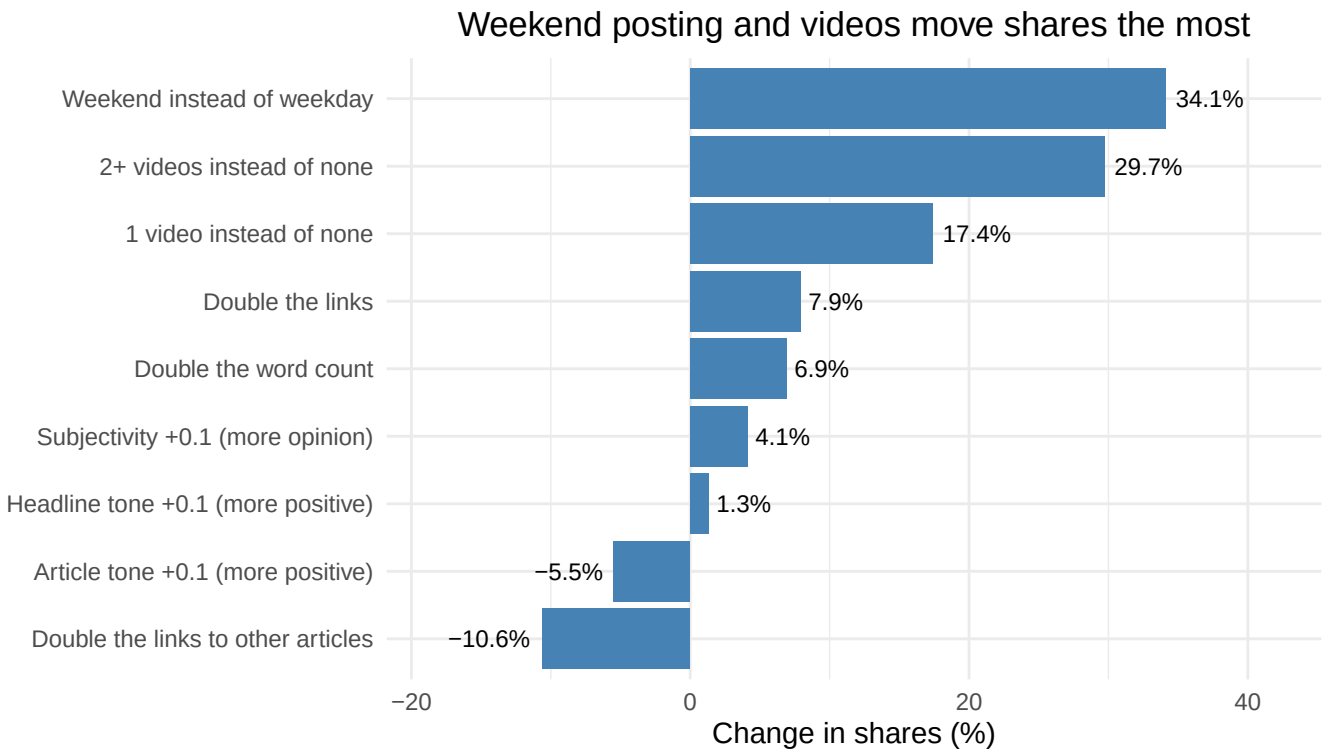
effects <- data.frame(
  change = c("Weekend instead of weekday", "1 video instead of none",
             "2+ videos instead of none", "Double the word count",
             "Double the links", "Double the links to other articles",
             "Article tone +0.1 (more positive)", "Subjectivity +0.1 (more opinion)",
             "Headline tone +0.1 (more positive)",
             "Double keyword popularity", "Double linked-article popularity"),
  group = c(rep("Controllable", 9), rep("Uncontrollable", 2)),
  pct_change = c(exp(b["weekendWeekend"]) - 1,
                 exp(b["vid_group1"]) - 1,
                 exp(b["vid_group2+"]) - 1,
                 2^b["log_words"] - 1,
                 2^b["log_hrefs"] - 1,
                 2^b["log_self_hrefs"] - 1,
                 exp(0.1 * b["global_sentiment_polarity"]) - 1,
                 exp(0.1 * b["global_subjectivity"]) - 1,
                 exp(0.1 * b["title_sentiment_polarity"]) - 1,
                 2^b["log_kw_avg_avg"] - 1,
                 2^b["log_self_ref"] - 1)
) |>
  mutate(pct_change = round(100 * pct_change, 1))

knitr::kable(effects, row.names = FALSE)
```

change	group	pct_change
Weekend instead of weekday	Controllable	34.1
1 video instead of none	Controllable	17.4
2+ videos instead of none	Controllable	29.7
Double the word count	Controllable	6.9
Double the links	Controllable	7.9
Double the links to other articles	Controllable	-10.6
Article tone +0.1 (more positive)	Controllable	-5.5
Subjectivity +0.1 (more opinion)	Controllable	4.1
Headline tone +0.1 (more positive)	Controllable	1.3
Double keyword popularity	Uncontrollable	9.2
Double linked-article popularity	Uncontrollable	3.0

```
effects |>
  filter(group == "Controllable") |>
  ggplot(aes(pct_change, reorder(change, pct_change))) +
  geom_col(fill = "steelblue") +
```

```
  geom_text(aes(label = paste0(pct_change, "%"),
                hjust = ifelse(pct_change > 0, -0.15, 1.15)), size = 3) +
  scale_x_continuous(expand = expansion(mult = 0.25)) +
  labs(x = "Change in shares (%)", y = NULL,
       title = "Weekend posting and videos move shares the most")
```



- Posting on a weekend instead of a weekday: about 34% more shares
- One video: about 17% more shares. Two or more: about 30% more
- Doubling the word count: about 7% more. Doubling the links: about 8% more
- Doubling links to other articles: about 11% fewer
- A more positive article: slightly fewer shares. A more opinionated article or a more positive headline: slightly more
- Images aren't in the final model, so they don't affect shares once the other features are accounted for

URL Date Check

```
g_url <- update(g_bic, . ~ . - weekend + url_is_weekend)
round(100 * (exp(coef(g_url)["url_is_weekend"]) - 1), 1)
```

```
## url_is_weekend
##           25.8
```

- About a quarter of articles have a different weekday in the URL than in the data
- Using the URL date, the weekend effect is about 26% instead of 34%
- Smaller, but still the largest controllable effect